# Clustering Algorithms

## K-Means & Hierarchical Clustering

Artificial Intelligence and Machine Learning |With IIT Ropar

NIELIT Ropar • Unsupervised Learning Module

Unsupervised Learning | Clustering | Scikit-Learn | Python | Data Science

# Agenda

01 **What is Clustering?**

Overview, types, and real-world use cases

02 **K-Means Clustering**

Algorithm, steps, elbow method, implementation

03 **K-Means Code Lab**

Full Python implementation with Scikit-Learn

04 **Hierarchical Clustering**

Agglomerative & divisive approaches, dendrograms

05 **Hierarchical Code Lab**

Dendrogram plotting & cluster extraction

06 **Comparison & Best Practices**

When to use which algorithm

# What is Clustering?

Unsupervised Learning

*Grouping data points so that items in the same group (cluster) are more similar to each other than to those in other groups - without using predefined labels.*

### Customer Segmentation

Group customers by purchase behaviour, demographics, and preferences for targeted marketing.

### Document Clustering

Organise news articles or search results into topics automatically (Google News, etc.).

### Anomaly Detection

Identify outliers in network traffic, fraud detection, and medical diagnosis.

### Image Compression

Reduce image colours by replacing similar pixels with a single representative centroid.

### Gene Expression Analysis

Cluster genes with similar expression patterns to discover biological pathways.

### Recommendation Systems

Group users or items into clusters to make collaborative-filtering recommendations.

# K-Means Clustering — Core Concept

Algorithm

**1**

**Initialise**

Randomly choose K data points as initial centroids.

**2**

**Assign**

Assign each data point to the nearest centroid using Euclidean distance.

**3**

**Update**

Recompute each centroid as the mean of all points assigned to it.

**4**

**Repeat**

Iterate steps 2–3 until centroids no longer change (convergence).

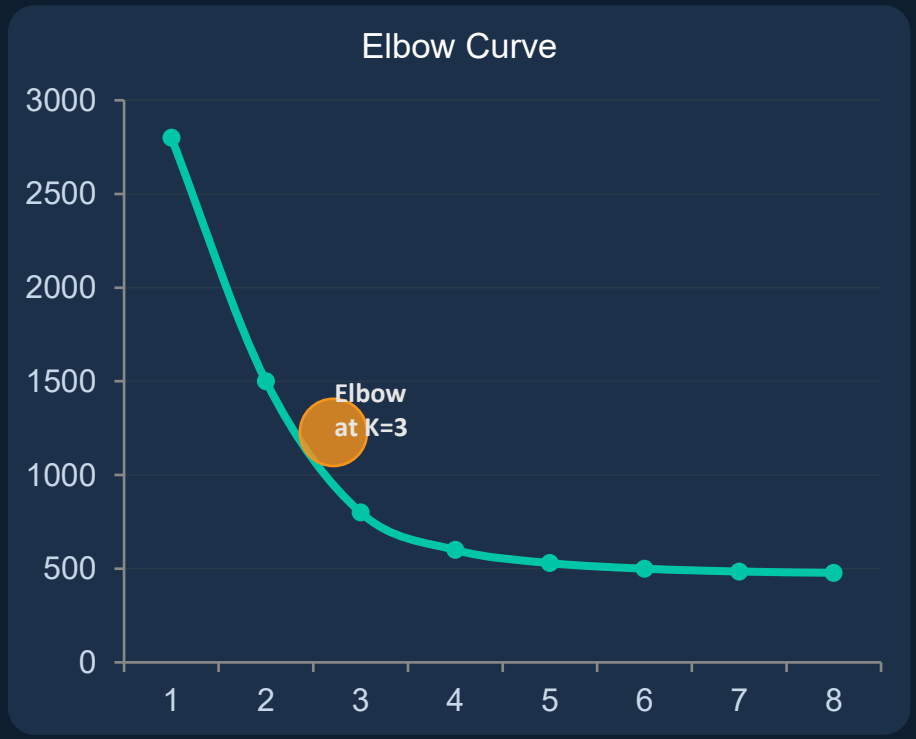
**Objective:** Minimise Within-Cluster Sum of Squares (WCSS) = $\Sigma \Sigma ||x_i - \mu_k||^2$ | Convergence guaranteed but may find local minimum → run multiple times (n_init)

# Choosing K — The Elbow Method

Hyperparameter

**How do we pick the right number of clusters?**

1. Run K-Means for K = 1, 2, 3, … n
2. Calculate WCSS (inertia) for each K
3. Plot K vs. WCSS on a line chart
4. Look for the 'elbow' — where WCSS drop flattens
5. That K value is the optimal number of clusters
6. Silhouette Score can validate the choice




**Silhouette Score**

Ranges from -1 to +1. Higher = better-defined clusters. Use sklearn.metrics.silhouette_score(X, labels).

# K-Means — Full Python Implementation

Code Lab

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

# Generate synthetic data
X, y_true = make_blobs(n_samples=300,
    centers=4, cluster_std=0.7, random_state=42)

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Elbow method
inertia = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, n_init=10,
random_state=42)
    km.fit(X_scaled)
    inertia.append(km.inertia_)
```

```
# Fit final model with K=4
kmeans = KMeans(n_clusters=4, n_init=10,
    random_state=42)
labels = kmeans.fit_predict(X_scaled)
centers = kmeans.cluster_centers_

# Silhouette score
from sklearn.metrics import silhouette_score
score = silhouette_score(X_scaled, labels)
print(f"Silhouette: {score:.3f}")

# Visualise clusters
colors =
['#00C6A7','#F7941D','#4EC9E5','#FF6B6B']
for i in range(4):
    mask = labels == i
    plt.scatter(X_scaled[mask,0],
        X_scaled[mask,1], c=colors[i], s=40)
plt.scatter(centers[:,0], centers[:,1],
    marker='X', s=200, c='black', zorder=5)
plt.title('K-Means Clusters')
plt.show()
```

# K-Means — Key Parameters & Output

Scikit-Learn

| Parameter | Default | Description |
|---|---|---|
| n_clusters | 8 | Number of clusters K to form |
| init | 'k-means++' | Initialisation method. k-means++ is smarter than random |
| n_init | 10 | Number of times algorithm runs with different centroid seeds |
| max_iter | 300 | Maximum iterations per run before stopping |
| random_state | None | Seed for reproducibility — always set for research |
| algorithm | 'lloyd' | 'lloyd' (standard EM) or 'elkan' (faster for dense data) |

## Key Output Attributes

**`.labels_`**

Cluster index for each data point (0 to K-1)

**`.cluster_centers_`**

Coordinates of cluster centroids — shape (K, n_features)

**`.inertia_`**

Sum of squared distances of samples to nearest centroid (WCSS)

**`.n_iter_`**

Number of iterations to converge in the best run

# Hierarchical Clustering — Core Concept

Algorithm

Builds a tree of clusters (dendrogram) **without pre-specifying K**. Two main strategies:

### Agglomerative (Bottom-Up) ↑

Start with each point as its own cluster. Merge the two closest clusters at each step. Most commonly used approach.

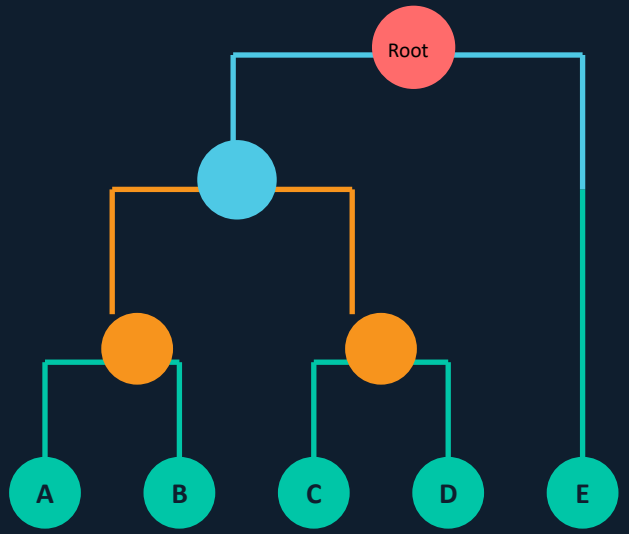
### Divisive (Top-Down) ↓

Start with all points in one cluster. Recursively split the cluster with the highest dissimilarity.

### Linkage Criteria (Distance Between Clusters)

| Single (min dist) | Complete (max dist) | Average (avg dist) | Ward (min variance) |
|---|---|---|---|

### Dendrogram (Tree of Merges)



# Hierarchical Clustering — Full Python Implementation

Code Lab

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import
AgglomerativeClustering
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram,
linkage

# Create & scale data
X, _ = make_blobs(n_samples=150, centers=4,
    cluster_std=0.8, random_state=42)
X = StandardScaler().fit_transform(X)

# Plot dendrogram using scipy
linked = linkage(X, method='ward')
plt.figure(figsize=(12, 5))
dendrogram(linked,
    orientation='top',
    distance_sort='descending',
    show_leaf_counts=True)
plt.title('Ward Linkage Dendrogram')
plt.show()
```

```python
# Fit AgglomerativeClustering
model = AgglomerativeClustering(
    n_clusters=4,
    linkage='ward',        # ward | average | complete | single
    affinity='euclidean'  # euclidean | cosine | manhattan
)
labels = model.fit_predict(X)

# Evaluate
from sklearn.metrics import (
    silhouette_score,
    calinski_harabasz_score,
    davies_bouldin_score
)
sil  = silhouette_score(X, labels)
ch   = calinski_harabasz_score(X, labels)
db   = davies_bouldin_score(X, labels)
print(f"Silhouette  : {sil:.3f}")
print(f"Calinski-H : {ch:.2f}")
print(f"Davies-Boul: {db:.3f}")

# Visualise
colors = ['#00C6A7','#F7941D','#4EC9E5','#FF6B6B']
for i in range(4):
    mask = labels == i
    plt.scatter(X[mask,0], X[mask,1],
        c=colors[i], label=f'Cluster {i}')
plt.legend()
plt.title(
'Hierarchical Clusters')
plt.show()
```

# Cluster Evaluation Metrics

Evaluation

## Silhouette Score

$$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

Range: −1 to +1 | +1 → perfectly clustered | 0 → on boundary | −1 → wrong cluster

## Calinski-Harabasz (CH) Index

$$CH = [\text{Between-SS} / (K-1)] / [\text{Within-SS} / (n-K)]$$

Range: 0 to ∞ | Higher is better — measures ratio of between-cluster to within-cluster dispersion

## Davies-Bouldin Index

$$DB = (1/K) \sum \max( (\sigma_i + \sigma_j) / d(c_i, c_j) )$$

Range: 0 to ∞ | Lower is better — average 'worst-case' similarity between each cluster and its most similar peer

These metrics are unsupervised — no ground-truth labels needed! For supervised comparison, also use Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI).

# K-Means vs Hierarchical Clustering

Comparison

| Aspect | K-Means | Hierarchical (Agglomerative) |
|---|---|---|
| Need to specify K | Yes — must define K upfront | No — cut dendrogram at any level |
| Scalability | Fast — $O(n \cdot K \cdot i \cdot d)$ — scales to millions | Slow — $O(n^2 \log n)$ — best for < 10k |
| Cluster Shape | Assumes spherical / convex clusters | Any shape (depends on linkage) |
| Result Type | Flat partition of K clusters | Dendrogram — full cluster hierarchy |
| Deterministic | No — sensitive to initialization | Yes — same result every run |
| Memory | Low — $O(K \cdot d)$ | High — $O(n^2)$ distance matrix |
| Best for | Large datasets, known cluster count | Small data, exploring hierarchy |
| Linkage methods | N/A (centroid-based) | Single, Complete, Average, Ward |

# Best Practices & Common Pitfalls

Tips

## ✓ Best Practices

- Always scale features (StandardScaler / MinMaxScaler) before clustering
- Run K-Means multiple times (n_init ≥ 10) to escape local minima
- Use Elbow + Silhouette together to confirm optimal K
- Use Ward linkage as default for Hierarchical — minimises variance
- Try DBSCAN / HDBSCAN if clusters are non-spherical or noisy
- Visualise with PCA/t-SNE after clustering for high-dimensional data

## ✗ Common Pitfalls

- Don't use K-Means on categorical data — use K-Modes or K-Prototypes
- Don't ignore outliers — they heavily distort centroids in K-Means
- Don't use single linkage in practice — prone to chaining effect
- Don't run Hierarchical clustering on very large datasets (> 10k) without sampling
- Don't treat inertia alone as the metric — it always decreases with more K
- Don't skip feature selection — noise features degrade cluster quality

# Summary

| | |
|---|---|
| **K-Means** | Centroid-based, fast, **requires K upfront**. Use **Elbow + Silhouette** for tuning. **Scale data first**. |
| **Hierarchical** | Dendrogram-based, no K required, reveals hierarchy. Ward linkage works best in most cases. |
| **Evaluation** | **Silhouette Score**, Calinski-Harabász, and Davies-Bouldin measure cluster quality without labels. |
| **Choice** | Large dataset → K-Means. Small dataset / need hierarchy → Agglomerative. Non-spherical → DBSCAN. |